HDOC DAY 63

Topic: Finding a missing integer using pointers

Aim

Find and display the one missing number from the range 0 through size, given size distinct array elements, using a pointer parameter and a separate function.

Algorithm

1. Start.
2. Read positive size and size distinct integers from 0 to size.
3. Pass the array address and size to missing_number().
4. Calculate expected_sum = size * (size + 1) / 2 using long long arithmetic.
5. Add all array elements using pointer arithmetic to obtain actual_sum.
6. Display expected_sum - actual_sum and stop.

Test Case Table

Test	Size	Input array	Expected result
1	3	3 0 1	Missing number: 2
2	2	0 1	Missing number: 2
3	9	9 6 4 2 3 5 7 0 1	Missing number: 8
4	1	0	Missing number: 1
5	1	1	Missing number: 0

C Program

```
#include <stdio.h>

void missing_number(int *nums, int size)
{
    long long expected_sum = (long long)size * (size + 1) / 2;
    long long actual_sum = 0;
    int i;
    for (i = 0; i < size; i++)
        actual_sum += *(nums + i);
    printf("Missing number: %lld\n", expected_sum - actual_sum);
}

int main(void)
{
    int size, i;
    printf("Enter size: ");
    if (scanf("%d", &size) != 1 || size <= 0) {
        printf("Invalid array size.\n");
        return 1;
    }
    int nums[size];
    printf("Enter array elements:\n");
    for (i = 0; i < size; i++)
        scanf("%d", &nums[i]);
    missing_number(nums, size);
```

```
    return 0;
}
```

Compilation and Execution (Mac Terminal)

```
clang -std=c99 day63.c -o day63
./day63
```

Sample Output

```
Enter size: 9
Enter array elements:
9 6 4 2 3 5 7 0 1
Missing number: 8
```

Result

The missing integer was determined successfully using pointers and a user-defined function. Time complexity: O(size).